SOMAIYA VIDYAVIHAR

K J Somaiya Institute of Technology

An Autonomous Institute Permanently Affiliated to the University of Mumbai

Name: Vaishnavi More
Roll Number: B - 72
Batch: SB4

Experiment No : 03

Program :

```
#include<stdio.h>
#include<conio.h>

char stack[50];
int top = -1;

void push(char ch)
{
     top++;
     stack[top] = ch;
}

char pop()
{
```

```
    char ch;

    ch = stack[top];
    top--;

    return ch;
}

int priority(char ch)
{
    if(ch == '+' || ch == '-')
        return 1;

    if(ch == '*' || ch == '/' || ch == '%')
```

```
        return 2;

    return 0;
}

void infix_to_postfix(char infix[], char postfix[])
{
    int i = 0, j = 0;
    char ch;

    while(infix[i] != '\0')
    {
        ch = infix[i];
```

```
        if((ch >= 'A' && ch <= 'Z') ||
           (ch >= 'a' && ch <= 'z') ||
           (ch >= '0' && ch <= '9'))
        {
            postfix[j] = ch;
            j++;
        }
        else if(ch == '(')
        {
            push(ch);
        }
        else if(ch == ')')
        {
            while(top != -1 && stack[top] != '(')
```

```
            {
                postfix[j] = pop();
                j++;
            }

            pop();
        }
        else
        {
            // pop operators with higher or equal priority
            while(top != -1 && priority(stack[top]) >= priority(ch))
            {
                postfix[j] = pop();
                j++;
```

```
            }

            push(ch);
        }

        i++;
    }

    while(top != -1)
    {
        postfix[j] = pop();
        j++;
    }
```

```
    postfix[j] = '\0';
}

void main()
{
    char infix[50], postfix[50];

    clrscr();

    while(1)
    {
        printf("\nenter infix expression (type exit to exit): ");
        scanf("%s", infix);
```

```
        if(infix[0] == 'e' &&
           infix[1] == 'x' &&
           infix[2] == 'i' &&
           infix[3] == 't' &&
           infix[4] == '\0')
        {
            printf("exited program\n");
            break;
        }

        infix_to_postfix(infix, postfix);

        printf("postfix expression: %s\n", postfix);
    }
```

```
    getch();
}
```

Output :

```
enter infix expression (type exit to exit): a-b+c*d
postfix expression: ab-cd*+

enter infix expression (type exit to exit): (a+b)/c*d
postfix expression: ab+c/d*

enter infix expression (type exit to exit): a+b-c
postfix expression: ab+c-
```